

Lab 4- Part 3

WHAT IS A TABLEVIEW?

A table shows data in rows and columns.

Example:

Name	Age	City
Rajit	25	Dhaka
Awal	30	Saidpur
Tanjir	28	Chittagong

STEP 1: Create Empty Table

Code:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.TableView;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class MyTable extends Application {

    @Override
    public void start(Stage primaryStage) {
        TableView table = new TableView();

        VBox root = new VBox(10, table);
        Scene scene = new Scene(root, 400, 300);

        primaryStage.setTitle("My Table");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

```
}  
}
```

What you see: Empty white box (table)

STEP 2: Add Columns

Code:

```
import javafx.application.Application;  
import javafx.scene.Scene;  
import javafx.scene.control.TableView;  
import javafx.scene.control.TableColumn;  
import javafx.scene.layout.VBox;  
import javafx.stage.Stage;  
  
public class MyTable extends Application {  
  
    @Override  
    public void start(Stage primaryStage) {  
        TableView table = new TableView();  
  
        TableColumn col1 = new TableColumn("Name");  
        TableColumn col2 = new TableColumn("Age");  
  
        table.getColumns().addAll(col1, col2);  
  
        VBox root = new VBox(10, table);  
        Scene scene = new Scene(root, 400, 300);  
  
        primaryStage.setTitle("Table with Columns");  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

What you see: Two columns: Name and Age (no data yet)

STEP 3: Create a Class to Hold Data

Add this class before your main class:

```
class Person {  
    String name;  
    int age;  
  
    Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

Why? Each row needs multiple pieces of data. Person class holds name and age together.

STEP 4: Connect Class to Table

Change your TableView line to:

```
TableView<Person> table = new TableView<>();
```

Change column lines to:

```
TableColumn<Person, String> col1 = new TableColumn<>("Name");  
TableColumn<Person, Integer> col2 = new TableColumn<>("Age");
```

STEP 5: Tell Columns Where to Get Data

Add this code after creating columns:

```
col1.setCellValueFactory(cellData -> {  
    Person p = cellData.getValue();  
    return new javafx.beans.property.SimpleStringProperty(p.name);  
});  
  
col2.setCellValueFactory(cellData -> {
```

```
        Person p = cellData.getValue();
        return new javafx.beans.property.SimpleIntegerProperty(p.age).asObject();
    });
```

What this means: "For Name column, take the name from Person object. For Age column, take the age."

STEP 6: Add Some Data

Add this code after `table.getColumns.addAll`:

```
ObservableList<Person> data = FXCollections.observableArrayList();
data.add(new Person("Rajit", 25));
data.add(new Person("Awal", 30));
data.add(new Person("Tanjir", 28));

table.setItems(data);
```

Add these imports at top:

```
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
```

COMPLETE WORKING CODE

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.TableView;
import javafx.scene.control.TableColumn;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;

class Person {
    String name;
    int age;

    Person(String name, int age) {
```

```

        this.name = name;
        this.age = age;
    }
}

public class MyTable extends Application {

    @Override
    public void start(Stage primaryStage) {
        TableView<Person> table = new TableView<>();

        TableColumn<Person, String> col1 = new TableColumn<>("Name");
        TableColumn<Person, Integer> col2 = new TableColumn<>("Age");

        col1.setCellValueFactory(cellData -> {
            Person p = cellData.getValue();
            return new javafx.beans.property.SimpleStringProperty(p.name);
        });

        col2.setCellValueFactory(cellData -> {
            Person p = cellData.getValue();
            return new
javafx.beans.property.SimpleIntegerProperty(p.age).asObject();
        });

        table.getColumns().addAll(col1, col2);

        ObservableList<Person> data = FXCollections.observableArrayList();
        data.add(new Person("Rajit", 25));
        data.add(new Person("Awal", 30));
        data.add(new Person("Tanjir", 28));

        table.setItems(data);

        VBox root = new VBox(10, table);
        Scene scene = new Scene(root, 400, 300);

        primaryStage.setTitle("Working Table");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {

```

```
        launch(args);  
    }  
}
```

What you see:

Name	Age
Rajit	25
Awal	30
Tanjir	28

STEP 7: Add New Rows Dynamically

Add TextFields and Button:

```
TextField nameField = new TextField();  
nameField.setPromptText("Enter name");  
  
TextField ageField = new TextField();  
ageField.setPromptText("Enter age");  
  
Button addButton = new Button("Add Person");  
  
addButton.setOnAction(e -> {  
    String name = nameField.getText();  
    int age = Integer.parseInt(ageField.getText());  
    data.add(new Person(name, age));  
    nameField.clear();  
    ageField.clear();  
});
```

Add to VBox:

```
VBox root = new VBox(10, table, nameField, ageField, addButton);
```

Now you can: Type name and age, click button, new row appears instantly.

FOR YOUR REGISTRATION FORM

Create User class with all fields:

```
class User {  
    String name;  
    String dob;  
    String gender;  
    String reservation;  
    String tech;  
    String qualification;  
    String location;  
  
    User(String name, String dob, String gender, String reservation,  
        String tech, String qualification, String location) {  
        this.name = name;  
        this.dob = dob;  
        this.gender = gender;  
        this.reservation = reservation;  
        this.tech = tech;  
        this.qualification = qualification;  
        this.location = location;  
    }  
}
```

Create TableView with 7 columns (follow same pattern as above)

In Register button, collect all data and add to table:

```
registerButton.setOnAction(e -> {  
    String name = nameField.getText();  
    String dob = dobPicker.getValue().toString();  
  
    String gender = maleRadio.isSelected() ? "Male" : "Female";  
  
    String reservation = yesRadio.isSelected() ? "Yes" : "No";  
  
    String tech = "";  
    if(javaCheck.isSelected()) tech = tech + "Java ";  
    if(dotnetCheck.isSelected()) tech = tech + "DotNet";  
  
    String qualification = qualCombo.getValue();  
    String location = locCombo.getValue();  
  
    userList.add(new User(name, dob, gender, reservation, tech, qualification,
```

```
location));  
});
```

SIMPLE RECAP

Step	What you did
1	Created empty table
2	Added columns
3	Created Person class
4	Connected class to table
5	Told columns where to get data
6	Added hardcoded data
7	Made it dynamic with button

PRACTICE TASKS

1. Add a third column "City" to the Person table
 2. Modify Person class to include city
 3. Add a TextField for city input
 4. Modify the add button to include city
-